

Rapport de projet — EcoPlate Edge

Du besoin au prototype

Version : 1.4 — 21 septembre 2026

Nature : projet personnel technique

Statut : MVP fonctionnel, benchmark Food-101 et documentation réalisés. La validation sur des photos prises en conditions réelles reste à mener.

Synthèse exécutive

J'ai conçu EcoPlate Edge autour d'une question simple : peut-on donner un premier repère climatique à partir d'une photo sans envoyer cette photo à un serveur ? Le POC utilise un modèle EfficientNet-Lite0 quantifié, exécuté directement dans le navigateur avec MediaPipe. L'utilisateur voit les propositions du modèle, peut les corriger et peut ajouter une courte description. Une règle séparée relie ensuite huit familles alimentaires à des profils AGRIBALYSE 3.2 pour afficher un niveau A-E, ou ? si le résultat est trop incertain.

J'ai d'abord évalué le modèle ImageNet officiel, puis un modèle spécialisé sur Food-101. Sur les 25 250 images du jeu de validation relabellisé, le modèle spécialisé atteint 54,27 % en top-1, 82,95 % en top-3 et 53,69 % en macro-F1, contre respectivement 15,28 %, 33,90 % et 17,02 % pour ImageNet. Il pèse 3,95 Mio. Ce résultat justifie le choix du modèle pour le POC, mais ne prouve pas son efficacité sur des photos prises par de vrais utilisateurs. C'est pourquoi la correction humaine, le texte et le rejet restent essentiels. Une campagne séparée de 30 photos est préparée, mais pas encore réalisée.

Le rapport suit les cinq axes demandés dans l'énoncé : analyse du besoin et du contexte, audit de la solution data, choix d'une solution technique, aide à la décision, puis suivi des délais, des coûts, des livrables et des performances.

1. Collecte des besoins et analyse du contexte

1.1 Contexte et rôle

EcoPlate Edge est un projet personnel réalisé pour la formation. Il ne répond pas à la commande d'un client réel et ne remplace aucun système existant. J'ai donc pris en charge l'ensemble du POC : cadrage, audit des données, choix techniques, développement, évaluation, documentation et déploiement.

Objectifs du POC

- produire une démonstration compréhensible en moins d'une minute ;
- exécuter réellement un modèle dans le navigateur, et pas seulement présenter une maquette ;
- protéger une image potentiellement personnelle ;
- éviter qu'un indicateur pédagogique soit pris pour une mesure scientifique ;
- déployer sans infrastructure payante, sur GitHub Pages ;
- conserver les décisions, résultats et limites dans des documents consultables.

Le périmètre impose un navigateur moderne, un hébergement statique, aucun backend et aucun service payant. Le modèle doit rester compact, l'interface utilisable sur mobile et au clavier, et les données environnementales doivent provenir d'une source unique. Le temps de réalisation avait été estimé entre 26 et 40 heures.

1.2 Collecte et analyse du besoin métier

Parties prenantes

Partie prenante	Attente	Implication
Utilisateur de démonstration	réponse rapide, privée et compréhensible	fournit image, texte et correction
Évaluateur de la formation	traçabilité du besoin, des choix et des résultats	consulte démo, rapport, code et métriques
Mentor / manager pédagogique	cohérence du sujet et des livrables	valide le cadrage, la carte mentale et le support
Porteur du projet	périmètre réalisable et résultat mesurable	cadre, développe, mesure et documente

L'ADEME et Google fournissent respectivement les données environnementales et le modèle de départ. Ils n'ont pas été consultés dans le cadre du projet.

Méthode de recueil

Comme il s'agit d'un projet personnel, je n'ai pas mené d'entretiens métier. J'ai construit le besoin à partir de l'énoncé, du scénario de démonstration et de trois contraintes : confidentialité de la photo, simplicité d'usage et honnêteté du résultat. J'ai ensuite transformé ce cadrage en backlog et en critères d'acceptation. Les mesures Food-101 sont volontairement séparées des performances encore inconnues sur des photos réelles.

Besoins hiérarchisés

ID	Besoin	Priorité	Critère d'acceptation
B01	Analyser une image sans backend	P1	aucune requête contenant l'image ; inférence effective en navigateur
B02	Montrer les prédictions et la confiance	P1	suggestions au-dessus d'un seuil réglable, classe originale conservée
B03	Signaler les cas incertains	P1	? sous le seuil, en cas d'ambiguïté ou sans correspondance
B04	Corriger la proposition	P1	validation, retrait, remplacement et ajout possibles
B05	Donner un repère A-E compréhensible	P1	résultat issu de la liste affichée, facteurs et limites visibles
B06	Ajouter un texte facultatif	P2	synonymes, accents, quantités simples et inconnus gérés localement
B07	Indiquer l'origine des informations	P2	badges vision, texte et utilisateur
B08	Sourcer les profils	P2	version, code AGB, unité, DQR, justification et limites
B09	Mesurer et tester	P2	tests automatisés, taille, latence par inférence, protocole 30 images
B10	Être accessible et responsive	P2	navigation clavier, focus, contraste, mobile, score non porté par la couleur seule
B11	Gérer des portions détaillées	P3	reporté ; seule une masse explicitement saisie pondère les repères
B12	Accélérer par WebGPU	P3	reporté après benchmark CPU/WASM

Contraintes éthiques, sécurité et déploiement

- minimisation : l'image n'est ni persistée ni transmise ;
- transparence : source, confiance, mapping et limites sont consultables ;
- contrôle humain : aucune prédiction n'est irréversible ;
- accessibilité : messages textuels, libellés explicites et focus visibles ;
- environnement : aucune promesse de certification ou de valeur exacte ;

- sécurité : types de fichier et taille validés, aucune donnée utilisateur injectée en HTML, aucune clé secrète et aucun service serveur.

2. Audit de la solution data et adéquation au besoin

2.1 Solution actuelle ou proposée

Il n'y avait pas de solution à auditer au démarrage. J'ai donc comparé les choix de la première maquette : Vite et TypeScript pour l'interface, ONNX ou TensorFlow.js pour le modèle, des profils JSON et un déploiement GitHub Pages. Les essais m'ont conduit à MediaPipe Tasks Vision avec EfficientNet-Lite0 int8. Le modèle contient ses métadonnées, le runtime prend en charge l'inférence et le site peut rester entièrement statique.

Flux implémenté :

```
Fichier local -> validation 12 Mo -> décodage -> recadrage 224 x 224
-> MediaPipe WASM CPU -> modèle Food-101 -> top 3 des 8 familles
                        ou référence ImageNet -> association à 8 familles
Texte -> normalisation accents/synonymes/grammes -> familles + inconnus
Vision + texte + utilisateur -> fusion et contradictions -> profils AGRIBALYSE
-> règle A-E ou ? -> explication, confiance, hypothèses et alternative
```

Le modèle, le moteur WASM, les règles d'association et les profils environnementaux sont livrés avec le site. Une base, une API ou un orchestrateur n'apporteraient rien à ce POC ; leur absence réduit à la fois le coût et la surface d'attaque.

2.2 Évaluation de l'adéquation aux besoins

Critère	Évaluation	État observé / écart
Confidentialité	couverte dans le périmètre	aucun envoi de photo, traitement en mémoire
Coût	cible atteinte	hébergement statique et inférence côté navigateur
Déploiement	réalisé	site Vite autonome et déploiement GitHub Pages automatisé
Latence	partiellement validée	médiane 31 ms et p95 59 ms dans Firefox headless ; appareils cibles non testés
Couverture alimentaire	incomplète	top-3 Food-101 à 82,95 % ; photos utilisateur non mesurées
Robustesse	incomplète	rejet et correction présents ; comportement réel à mesurer
Explication du résultat	implémentée	propositions, provenance, contradictions et règles visibles
Maintenance	mécanismes présents	modules typés, JSON versionné et tests automatisés ; maintenance en production non évaluée
Accessibilité	partielle	clavier et focus prévus ; audit manuel restant
Passage à l'échelle	adapté au site statique	calcul effectué sur chaque appareil, sans charge serveur

Audit des données visuelles

Food-101 contient 101 catégories de plats. Pour entraîner un modèle à huit sorties, j'ai dû les regrouper en huit familles. Ce choix déséquilibre les classes et force 61 recettes ambiguës dans une famille unique. Une seconde évaluation les marque donc comme `unknown`. ImageNet, avec ses 1 000 catégories généralistes, sert uniquement de point de comparaison. La validation en situation réelle dépend toujours des 30 photos prévues.

Audit des profils environnementaux

J'ai extrait huit produits de référence d'AGRIBALYSE 3.2, depuis le même fichier et le même indicateur. La difficulté est qu'un produit précis ne représente pas toute une famille alimentaire. Les valeurs servent donc seulement à construire un repère A-E ; elles ne sont pas affichées comme l'empreinte du repas. La version, la licence, les DQR et les limites sont conservées dans la fiche des données.

Ce qu'il reste à vérifier

- collecter et exécuter les 30 photographies autorisées ;
- mesurer médiane/p95 sur ordinateur et mobile ;
- vérifier manuellement réseau, contraste et lecteurs d'écran ;
- compléter l'évaluation avant toute généralisation à des photos utilisateur.

3. Identification de la solution technique

3.1 Comparatif des approches

Local ou backend

Option	Données personnelles	Coût récurrent	Taille du modèle	Complexité	Décision
Backend d'inférence	transfert à sécuriser	moyen	peu limitée	élevée	non retenu pour le POC
API tierce	image transmise	variable	peu limitée	moyenne	non retenue
Navigateur WASM	image conservée localement	nul	limitée	faible	retenu

Modèle généraliste ou spécialisé

Option	Taille	Couverture aliments	Provenance	Risque
EfficientNet-Lite0 int8 / ImageNet	5,18 Mio	top-3 33,90 % sur Food-101	officielle et claire	correspondances très incomplètes
EfficientNet-Lite0 spécialisé / Food-101	3,95 Mio	top-3 82,95 % sur Food-101	entraînement local documenté	regroupement forcé, comportement réel inconnu
SlimSAM + classifieur	poids supplémentaires	gain non établi	ablation seulement	latence et téléchargement

Qualitatif ou chiffré

Option	Compréhension	Risque de fausse précision	Données nécessaires	Décision
kg CO2e automatique	élevée en apparence	très élevé	masse et recette fiables	rejetée
A-E avec limites	élevée	maîtrisable	famille + repère	retenue
aucun résultat climat	faible valeur produit	nul	aucune	rejetée

3.2 Matrice de décision pondérée

Notation de 1 (défavorable) à 5 (favorable).

Critère	Poids	Référence ImageNet	Modèle spécialisé Food-101	Service d'inférence spécialisé
Confidentialité	25 %	5	5	2
Taille / démarrage	20 %	4	5	4
Adéquation alimentaire	20 %	1	4	5
Coût	15 %	5	5	2
Maintenance	10 %	4	4	2
Faisabilité délai	10 %	5	4	2
Score pondéré / 5	100 %	3,90	4,60	3,00

J'ai retenu le modèle Food-101 parce qu'il obtient de meilleurs résultats qu'ImageNet en top-1, top-3 et macro-F1, tout en étant plus petit. J'ai aussi conservé l'image entière comme prétraitement par défaut. Sur l'ablation de 808 images, la grille gagne 4,83 points en top-1, mais perd 0,12 point en top-3 et multiplie la latence par environ huit. SlimSAM n'a pas pu être évalué : le chargement de ses poids a échoué avant la première inférence. Je l'ai donc retiré de l'interface au lieu de supposer un gain.

3.3 Architecture cible et sécurité

L'architecture détaillée figure dans docs/architecture.md . Les modules échangent des objets TypeScript en mémoire : le calcul climatique reçoit des familles alimentaires, jamais l'image ni un tenseur. Le workflow de déploiement ne demande que les droits nécessaires à GitHub Pages (contents: read , pages: write et id-token: write). Aucun secret applicatif n'est utilisé.

3.4 Identification et priorisation des cas d'usage

Cas	Valeur	Effort	Priorité
Aliment isolé + correction	5	3	P1
Description seule	4	2	P1
Plat simple + texte	4	3	P1
Assiette multi-aliments	5	5	futur
Portion déclarée	3	2	P2 partielle
Comparaison de repas	3	3	futur
Historique utilisateur	2	4	exclu

4. Appui à la décision et méthode de mise en œuvre

4.1 Proposition de démarche projet

Roadmap, jalons et responsabilités

J'ai réalisé seul les différentes phases. Le mentor intervient pour valider le sujet, la carte mentale et le support du portfolio.

Phase	Contenu et outils	Estimation	Résultat / état
0	cadrage, backlog, critères	1-2 h	plan et périmètre — réalisé
1	faisabilité MediaPipe / TFLite	1-2 h	référence ImageNet de 5,18 Mio — réalisé
2-3	TypeScript, Vite, vision locale	5-9 h	modules, trois meilleures réponses, associations et erreurs — réalisé
4-6	profils JSON, texte, fusion	8-14 h	données sourcées, correction et provenance — réalisé
7	score et interface	4-6 h	A-E / ? , interface responsive, clavier et focus implémentés ; audit WCAG et essais mobiles restant à mener
8	Vitest et scripts de benchmark	4-6 h	Food-101 et ablation réalisés ; photos réelles à exécuter
9	Markdown, GitHub Actions / Pages	3-5 h	rapport, carte, workflow et pages — réalisé

Ces durées viennent du cadrage initial. Je n'ai pas suivi le temps réellement passé, ce qui empêche de comparer le prévu et le réalisé. Dans un contexte professionnel, j'ajouterais ce suivi dès le début.

Méthode

J'ai utilisé un Kanban simple : À faire , En cours , À vérifier , Terminé . Une tâche était considérée comme terminée lorsqu'elle disposait d'un test, d'une mesure, d'une source ou d'une limite écrite. La chaîne d'intégration installe les dépendances, exécute les tests, compile TypeScript, construit le site puis le publie.

Critères de fin

- code compilé sans erreur TypeScript ;
- tests automatisés passants ;
- classe originale et confiance visibles ;
- correction complète possible au clavier ;
- score issu de la liste visible ;
- source/licence/limites documentées ;
- build statique charge modèle et WASM sous un sous-chemin ;
- métriques non remplies remplacées par un état explicite, jamais une estimation.

4.2 Aide à la prise de décision

Risques et atténuations

Risque	Probabilité	Impact	Réponse	Indicateur
--------	-------------	--------	---------	------------

Risque	Probabilité	Impact	Réponse	Indicateur
classes difficiles à associer	forte	fort	correspondance limitée, ? , texte, modèle spécialisé si top-3 < 60 %	top-3, corrections
modèle trop lourd	faible	moyen	int8 3,95 Mio, chargement à la demande	taille, temps de chargement
photo complexe mal interprétée	forte	fort	périmètre visible, validation, cas hors périmètre	rejet utile
repère compris comme une mesure exacte	moyen	fort	vocabulaire indicatif, facteurs non affichés en résultat	test utilisateur
données environnementales mal interprétées	moyen	fort	source unique, codes, DQR, limites	tests de traçabilité
incompatibilité mobile	moyen	fort	CPU/WASM, essais sur plusieurs appareils	échecs, p95
dépendance fournisseur	faible	moyen	modèle local, abstraction model.ts	capacité de remplacement
accessibilité insuffisante	moyen	moyen	HTML natif, clavier, focus, audit manuel	problèmes constatés

Scénarios budgétaires

Pour comparer les scénarios, j'ai utilisé un taux indicatif de **450 € par jour** de 8 heures. Ce calcul donne un ordre de grandeur ; ce n'est pas un devis.

Scénario	Infrastructure	Travail estimé	Budget initial estimé	Usage recommandé
MVP statique Pages	0 € hors quotas	26–40 h	1 460–2 250 €	retenu
MVP + validation terrain	0 € hors appareils disponibles	+16–24 h	+900–1 350 €	prochaine étape
Service backend supervisé	10–50 € / mois au démarrage	10–15 j	4 500–6 750 €	seulement si Edge ne suffit plus

Critères de succès et état de mesure

Indicateur	Cible	État au 21 septembre 2026
Image transmise à un service d'inférence	0	architecture sans upload ; contrôle réseau manuel à maintenir
Top-3 sur cas en périmètre	≥ 60 %	82,95 % sur Food-101 ; non mesuré sur les 30 photos prévues
Rejet utile sur cas hors périmètre	≥ 70 %	non mesuré ; cas à collecter
Cas textuels documentés reconnus	≥ 90 %	15/15 passent ; échantillon trop petit pour estimer la couverture du langage
Latence médiane après chargement	< 500 ms ordinateur ; < 1 500 ms mobile	31 ms dans Firefox headless ; mobile non mesuré
Blocage clavier	0	aucun blocage identifié lors des essais préliminaires ; audit complet non terminé
Profils avec code, version, unité, DQR et limites	100 %	8/8
Démonstration comprise sans accompagnement	< 1 minute	non mesuré avec une personne extérieure

5. Contrôle des délais, coûts, livrables et

performances

5.1 Tableau de bord de pilotage

Dimension	Indicateur	Situation actuelle	Action
Délais	phases terminées / prévues	technique et documentation construits ; campagne réelle restante	planifier la collecte et l'évaluation
Coût	services payants	0 service requis	surveiller limites Pages
Livrables	critères de fin	application, tests, documentation, carte et automatisation présents	maintenir les liens et les versions
Données	profils traçables	8/8	revoir à chaque version AGRIBALYSE
Qualité texte	cas documentés	15 automatisés	enrichir seulement avec erreurs réelles
Vision Food-101	top-1 / top-3 / macro-F1	54,27 % / 82,95 % / 53,69 %	analyser les familles faibles
Vision en usage réel	top-1 / top-3 / rejet	mesures en attente	exécuter les 30 photos
Exécution locale	taille du modèle	4 140 006 octets	acceptable pour le prototype
Latence	médiane / p95	Firefox headless : 31 / 59 ms, 10 passages	mesurer sur appareils cibles
Accessibilité	blocage clavier	audit manuel non terminé	vérifier sur ordinateur et mobile

Le rejet utile, la latence mobile, l'accessibilité et la compréhension de la démo en une minute ne sont pas encore validés. Ils seront mesurés avec les 30 photos et au moins un utilisateur extérieur. Les 15 cas textuels passent, mais cet échantillon est trop petit pour conclure à une bonne couverture du langage naturel.

5.2 Outils et processus de suivi

Le prototype n'envoie aucune télémétrie. Il affiche uniquement la durée de la dernière inférence. L'utilisateur peut choisir d'exporter les résultats, qui sont ensuite agrégés avec `src/evaluation/metrics.ts`.

Vitest couvre les règles métier et les associations de classes. La compilation TypeScript et la construction du site Vite sont également contrôlées. Les vérifications manuelles portent sur le parcours, le réseau et l'affichage. Comme il n'y a pas de serveur applicatif, un test de charge HTTP serait peu pertinent ; il faut surtout mesurer le chargement, la mémoire et la latence sur plusieurs appareils.

Critères de contrôle en CI :

```
npm ci -> npm test -> npm run build -> artefact Pages -> déploiement
```

6. Conclusion et recommandations

EcoPlate Edge fonctionne comme prototype de démonstration : l'inférence se fait dans le navigateur, les propositions restent corrigeables et le repère climatique est calculé par une règle distincte. Le projet m'a surtout appris à ne pas confondre la sortie d'un modèle avec une décision fiable : il faut montrer l'incertitude, permettre la correction et documenter l'écart entre une famille alimentaire et un profil ACV précis.

La prochaine étape est d'exécuter le protocole de 30 photos sur ordinateur et mobile, puis de terminer l'audit d'accessibilité. Si le top-3 attendu n'est pas atteint, je réduirai le nombre de familles ou je réentraînerai le modèle avec de vraies classes `mixed_dish` et `unknown`. Les fonctions supplémentaires — quantités avancées, WebGPU ou comparaison de repas — ne sont utiles qu'après cette validation.

7. Ressources associées

- portfolio et démonstration en ligne : <https://soct.github.io/Ecoplate/> ;
- modèle EcoPlate utilisé par la démonstration : https://soct.github.io/Ecoplate/models/efficientnet_lite0_food101_8_int8.tflite ;
- jeu de données utilisé pour l'entraînement et l'évaluation : <https://huggingface.co/datasets/ethz/food101> ;
- page originale de Food-101 : https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/.